

Pinakes: Motor de Busca Wikipedia

Arquitetura Distribuída com Elasticsearch 8 e Spring Boot 3

Apresentação Técnica — Deep Dive Atômico

Arquitetura de Sistemas de Busca

30 de abril de 2026

Agenda do Mergulho Técnico

- 1 Infraestrutura e Bootstrap
- 2 Contrato e API Design
- 3 Arquitetura do Backend
- 4 Elasticsearch Deep Dive
- 5 Frontend e UX
- 6 Considerações Finais

Automação de Ambiente: O Makefile

O projeto utiliza um Makefile robusto para garantir reprodutibilidade total:

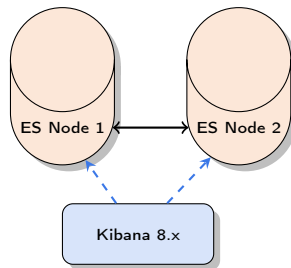
- **Gestão de JDK:** O comando `make java-install` configura o JDK 17 via symlinks (`~/java/current`), permitindo alternar versões sem afetar o sistema.
- **Orquestração Docker:** `make infra-up` não apenas sobe os serviços, mas executa um *health check* contínuo até que o cluster esteja operacional.
- **Carga de Dados:** `make seed` automatiza o processo de ingestão via NDJSON e scripts de carga dedicados.

Eficiência

Com um único comando (`make all`), o desenvolvedor sai de um ambiente vazio para uma aplicação funcional.

Componentes do Docker Compose:

- **Elasticsearch Cluster:** 2 nós configurados para alta disponibilidade.
- **Kibana:** Porta 5601. Essencial para análise de índices e execução de queries via *Dev Tools*.
- **Rede Isolada:** Bridge network customizada para comunicação inter-containers.



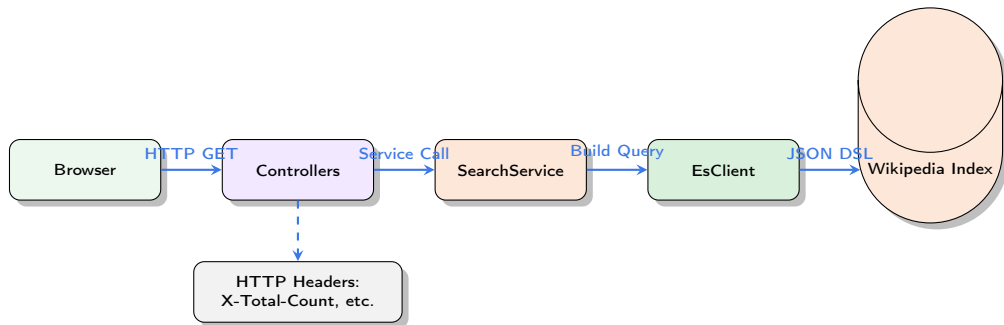
Design de API: O Contrato api.yml

A abordagem **API-First** garante que o backend e o frontend sigam o mesmo esquema.

```
paths:
  /search:
    get:
      summary: 'Busca_full-text_com_tuning_de_parametros'
      parameters:
        - name: query (string, required)
        - name: page (integer, default: 1)
        - name: fuzziness (string, default: AUTO)
        - name: phraseBoost (number, default: 2.0)
        - name: titleBoost (number, default: 1.5)
        - name: slop (integer, default: 0)
```

Os modelos e interfaces são gerados automaticamente durante a fase de compilação.

Fluxo de Dados: Do Browser ao Disco



SearchViewController (MVC)

- Rota: GET /
- Responsabilidade: Renderização via Thymeleaf.
- Popula o Model do Spring com resultados e sugestões.

SearchController (REST)

- Rota: GET /v1/search
- Responsabilidade: Dados puros (JSON).
- Implementa a interface SearchApi gerada.
- Injeta metadados de paginação nos **Headers**.

O SearchService atua como o orquestrador da lógica de negócio:

- **Conversão de Tipos:** Traduz o `ObjectNode` (JSON genérico) do driver ES para a classe `Result` (POJO).
- **Fallback de Destaque:** Caso o Elasticsearch não retorne *highlights*, o serviço sanitiza o conteúdo original usando **Regex** para evitar ataques XSS e remover lixo de formatação.
- **Agregação de Sugestões:** Coleta a primeira opção de cada token sugerido pelo motor de correção ortográfica.

A integração de baixo nível utiliza o *Elasticsearch Java API Client*:

- **Segurança (SSL/Auth):** Configura o `RestClientTransport` com suporte a HTTPS e credenciais básicas (`elastic/user123`).
- **Abstração de Query:** Centraliza a construção da Query Booleana, isolando a complexidade da sintaxe JSON da lógica de negócio.
- **Connection Management:** Gerencia o ciclo de vida do cliente e garante a correta liberação de recursos.

Mapeamento atômico dos campos:

- title: Tipo *text* com analisadores customizados para busca parcial.
- content: Tipo *text*, campo principal para busca *full-text* e *highlighting*.
- url: Tipo *keyword*, usado para links diretos sem análise textual.
- dt_creation: Tipo *date*, permite filtros temporais.
- reading_time: Tipo *integer*, metadado para enriquecimento da UX.

Estratégia de Relevância: Boolean Query

A consulta combina múltiplos fatores para garantir o melhor resultado:

- **must**: Correspondência textual no campo `content` com **Fuzziness: AUTO** (algoritmo Levenshtein).
- **should (Phrase)**: Aumenta o score se os termos aparecerem na ordem exata (*match_phrase*) com multiplicador de peso (*boost*).
- **should (Title)**: Atribui relevância extra se o termo for encontrado no título do artigo.

```
.must(m -> m.match(mq -> mq.field("content").query(query).fuzziness("AUTO")))  
.should(sh -> sh.matchPhrase(mp -> mp.field("content").query(query).boost(2.0f)))
```

Potencialidades Implementadas e Ativadas

- **Term Suggest:** Sugestões de correção ortográfica (“Você quis dizer?”).
- **Highlighting:** Tags `<strong>` injetadas dinamicamente.
- **Source Filtering:** Payload reduzido (includes/excludes).
- **Custom Analyzers:** *Asciifolding*, *Lowercase*, *Snowball*.
- **BM25 Ranking:** Algoritmo moderno de pontuação de relevância.
- **Ingest Pipelines:** Limpeza automática de dados na carga.

A interface é modularizada para máxima manutenibilidade:

- `search-bar.html`: Centraliza o input, ícones e dropdown de autocomplete.
- `result-item.html`: Template único para cada card de resultado, garantindo consistência visual.
- `pagination.html`: Lógica de navegação isolada.
- `sidebar.html`: Menu lateral com conceitos de curso integrados.

- **Autocomplete com Debounce:** Evita flooding de requests ao servidor (aguarda 300ms de inatividade).
- **API Service:** Abstração centralizada para chamadas via `fetch`.
- **Theme Manager:** Gerencia persistência de tema (Dark/Light) via *Local Storage* e *Media Queries* do sistema.
- **Accessibility Panel:** Ajuste dinâmico de fontes e contraste via variáveis CSS.

Conclusão: O Valor do Sistema Pinakes

- ❶ **Controle Total:** Parâmetros de tuning expostos para ajuste fino.
- ❷ **Robustez:** Infraestrutura resiliente e automatizada via Docker.
- ❸ **UX Moderna:** Autocomplete, sugestões e acessibilidade.
- ❹ **Escalabilidade:** Pronto para crescer com o volume de dados da Wikipedia.

Obrigado!